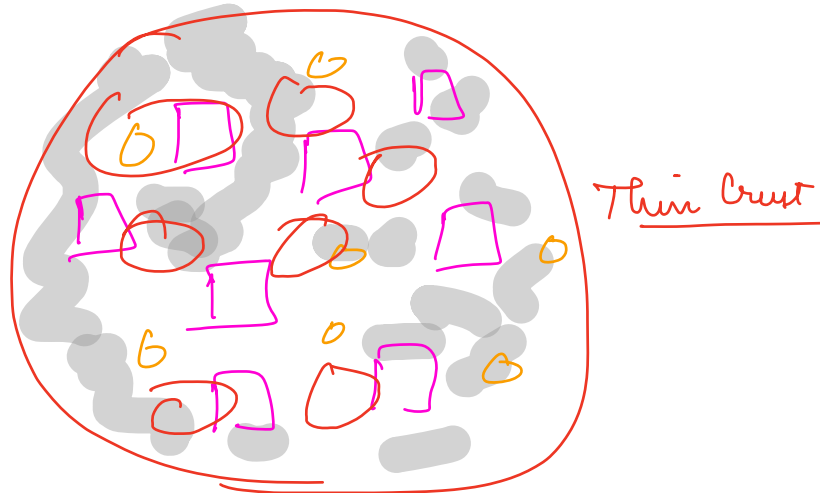


Agenda: Structural Design Pattern

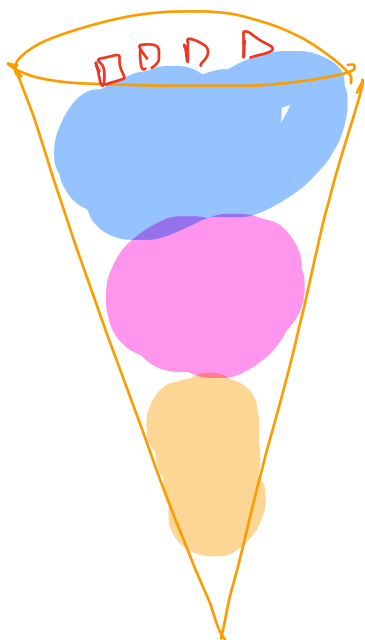
- ① Decorator Design Pattern
 - ② Flyweight Design Pattern
- Easy

DECORATOR DESIGN PATTERN

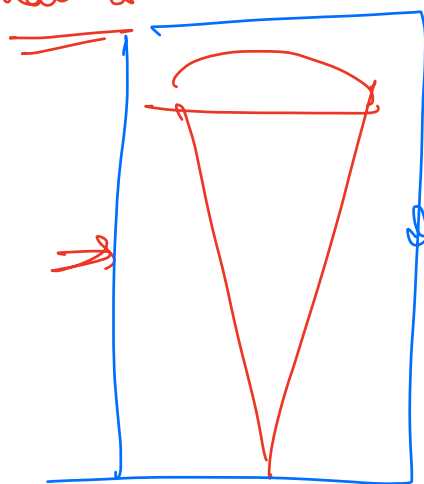
→ SWE at Domino's / Ice Cream / Coffee House
→ Order management system



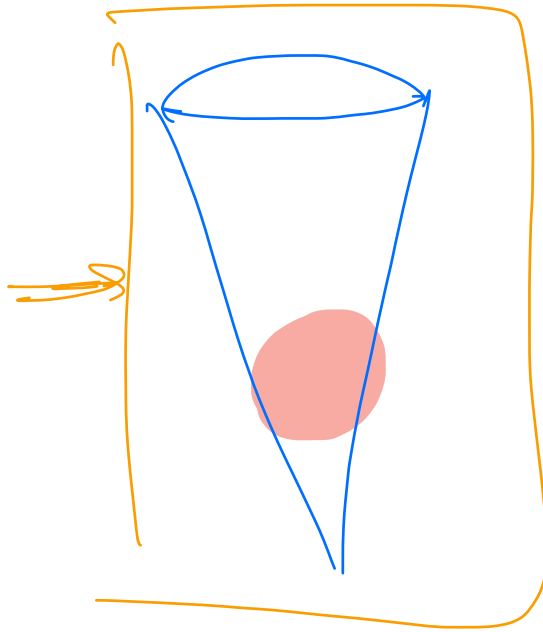
Ice Cream Cone



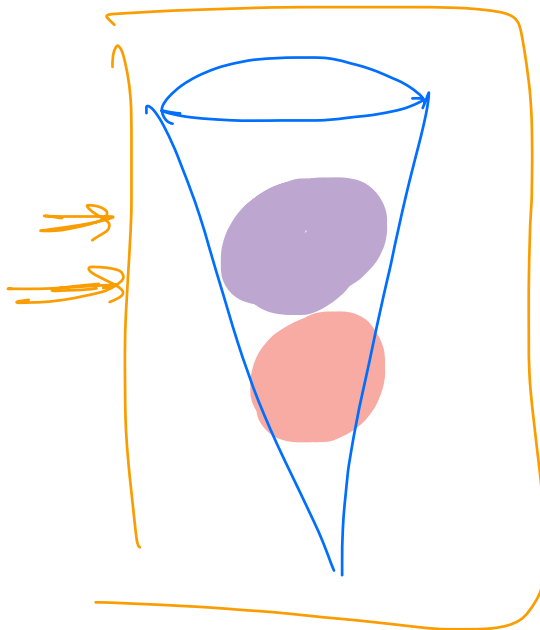
When we were ordering a cone
What is:



Ice Cream Cone

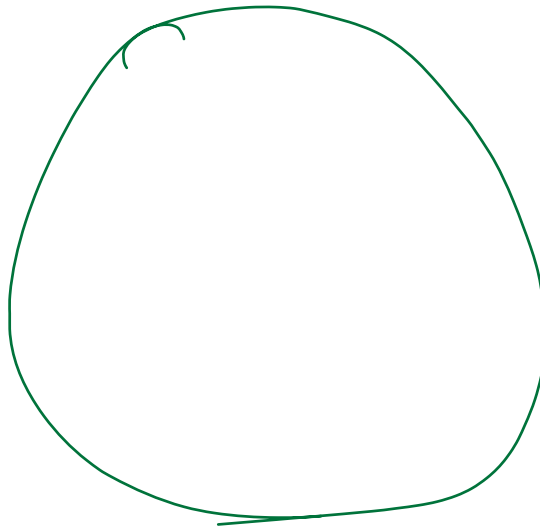


Ice Cream Cone

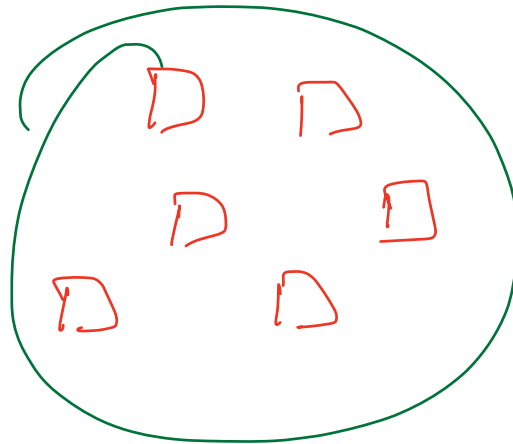


Ice Cream Cone

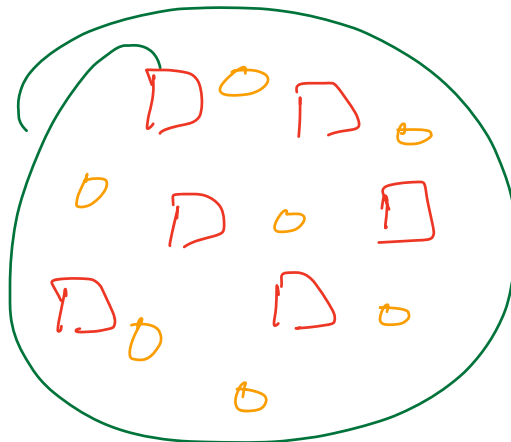
Ice Cream Cone $\Rightarrow \Rightarrow$ configuring $\Rightarrow \Rightarrow$ Ice Cream
Cone
 $\Rightarrow$ (higher price,
diff name)



Pizza Base

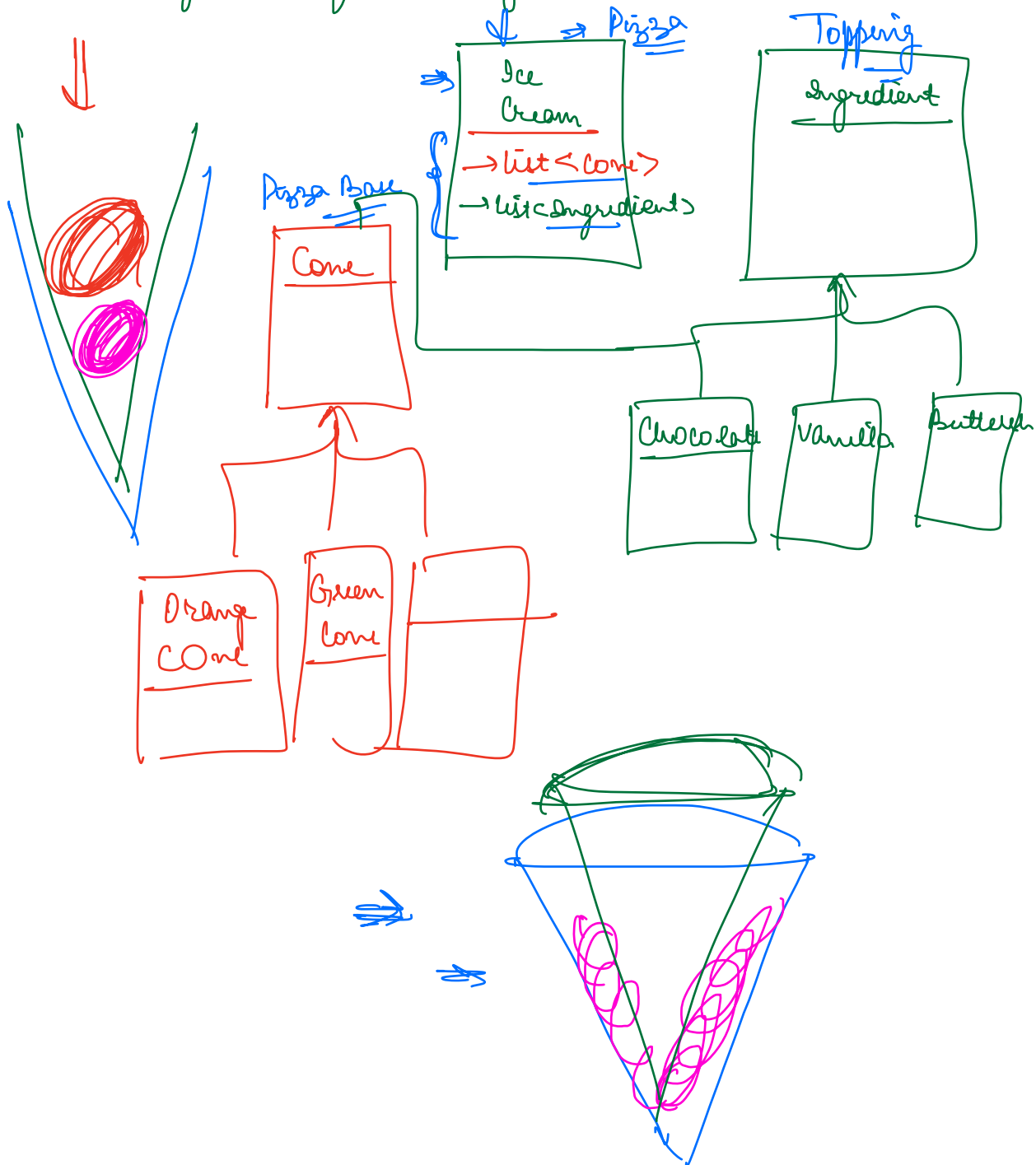


→ Pizza



→ Pizza

Build a LW system that takes Pizza orders
Ice Cream Cone orders. Make the class
diagram of the system.



⇒ Our design is unable to account for

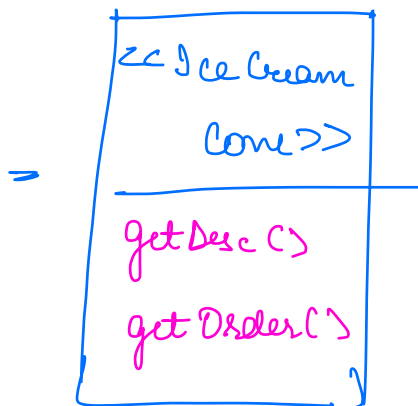
diff orders.

Decorator Design Pattern

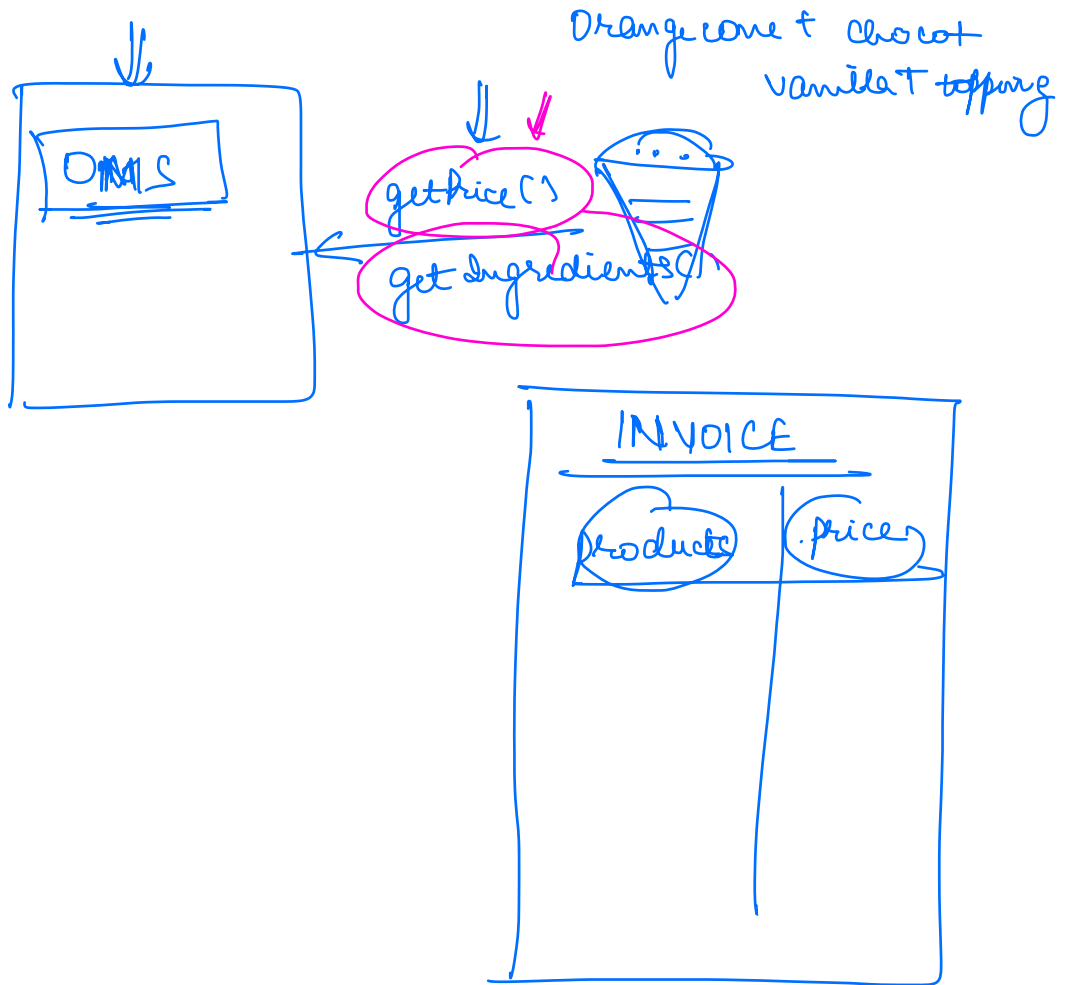
→ often we have a situation where we have to build something (X). To build that thing, we start from a base thing, we keep adding add-ons till we get to final mature thing.

(Ice Cream, Pizza, Coffee)

Step 1: For the thing that you are building,
Create an interface for that.

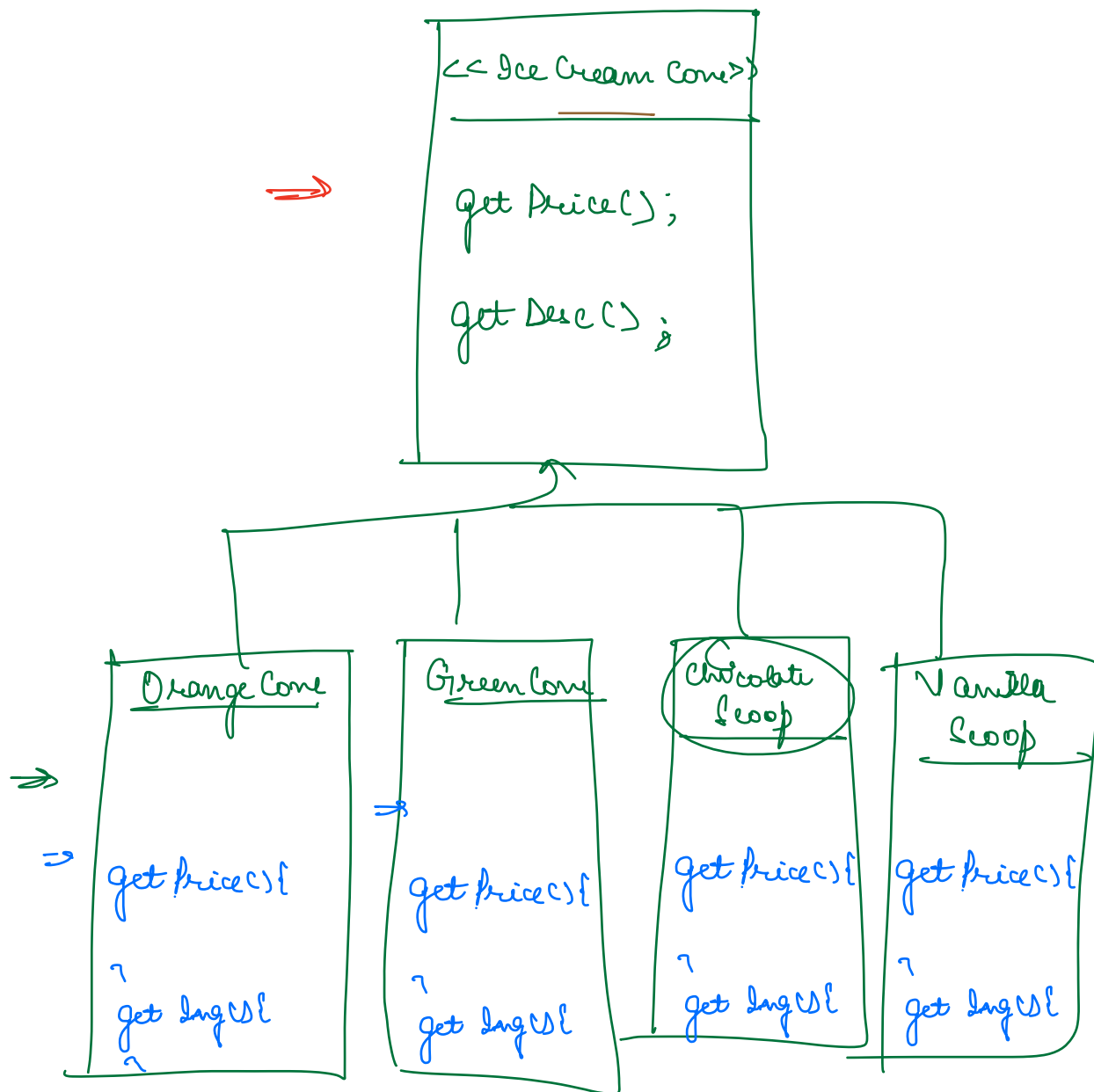


Step 2: Put methods that we expect from
an ice cream cone. ↗ Client expects



Step 2: For every constituent of ice cream cone
make it implement our interface.

- Base Constituents
- Toppings.

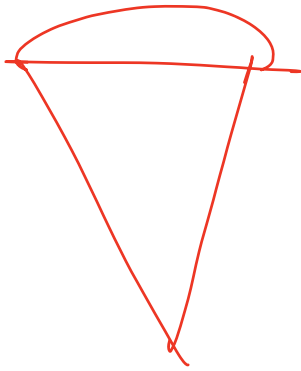


Base Ingredients

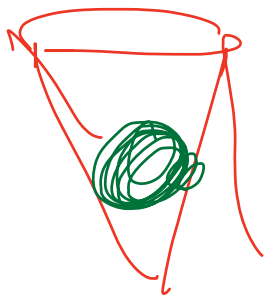
(assume a cone
can't be put
inside another
cone)

```

classDiagram
    class OrangeCone {
        +get Price()
        +get Ingredients()
    }
    OrangeCone {
        +get Price() {
            // whatever is price of cone
            return 10;
        }
        +get Ingredients() {
            return "Orange Cone"
        }
    }
  
```

⇒ Orange Cone: 10



⇒ Orange + Butterworth Cone : 25
price
Ingredients

class Chocolate Scoop implements Ice Cream Cone {
private IceCreamCone cone;

⇒ { Chocolate Scoop Ice Cream Cone cone {
this.cone = cone;
}

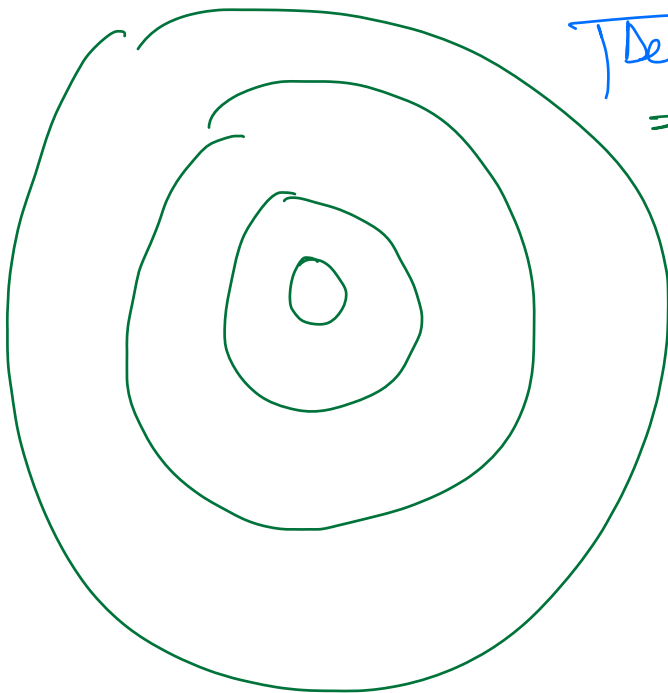
getPrice() {

return 40 + cone.getPrice();

}

getIngredients() {

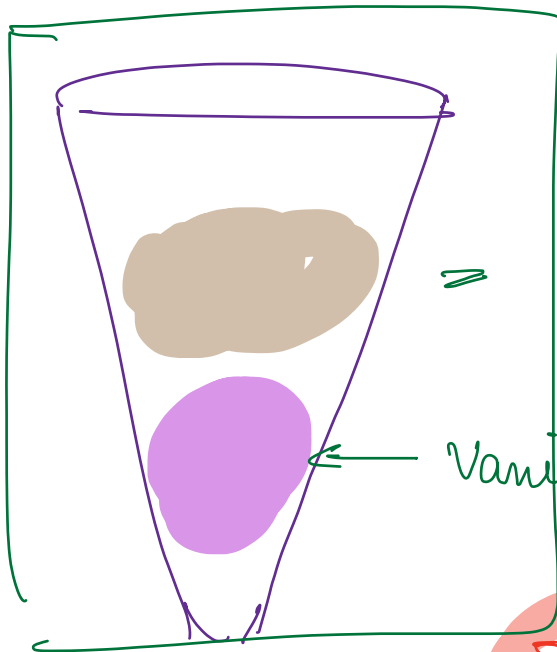
return cone.getIngredients() + "Chocolate";
}



Decorator == Wrapper

⇒ think of adding an add on as

wrapping a topping
over earlier thing



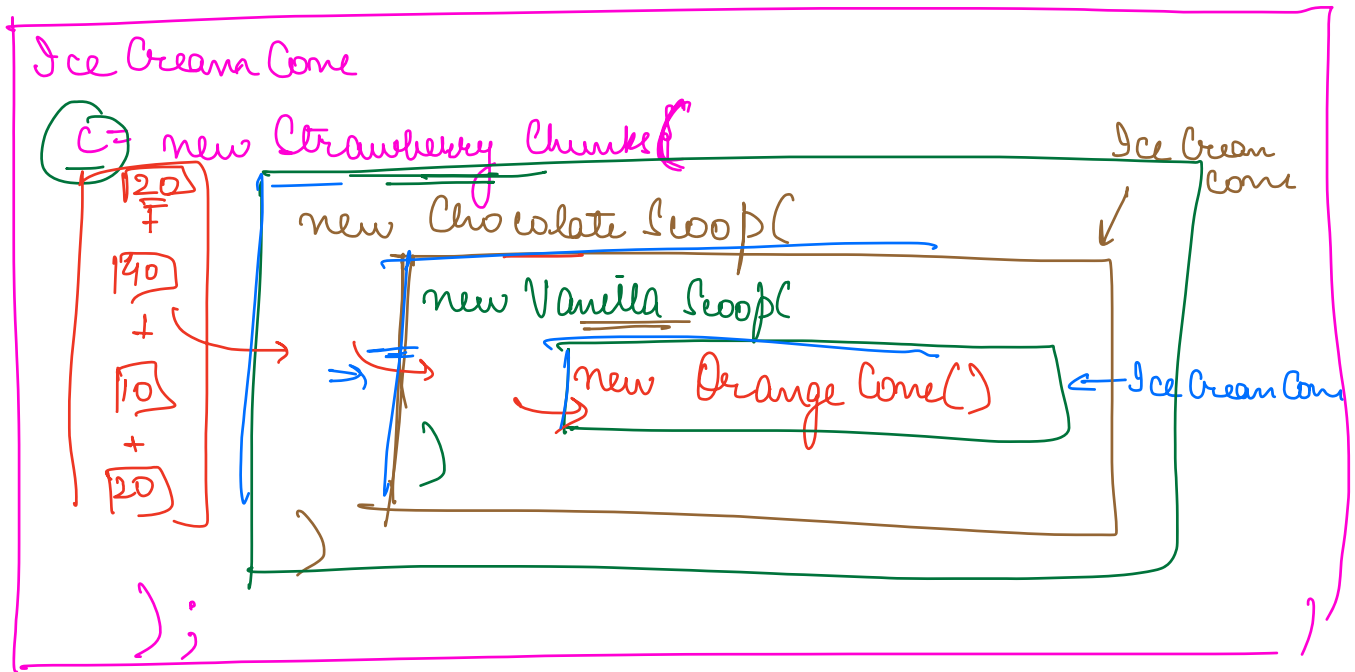
⇒ Ice Cream Cone

← Vanilla

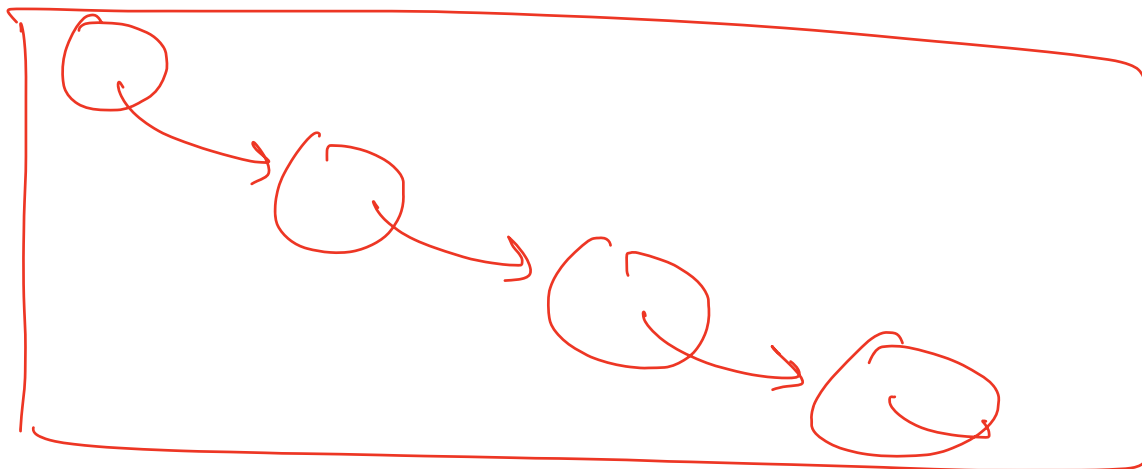
£ 50

+ (40)

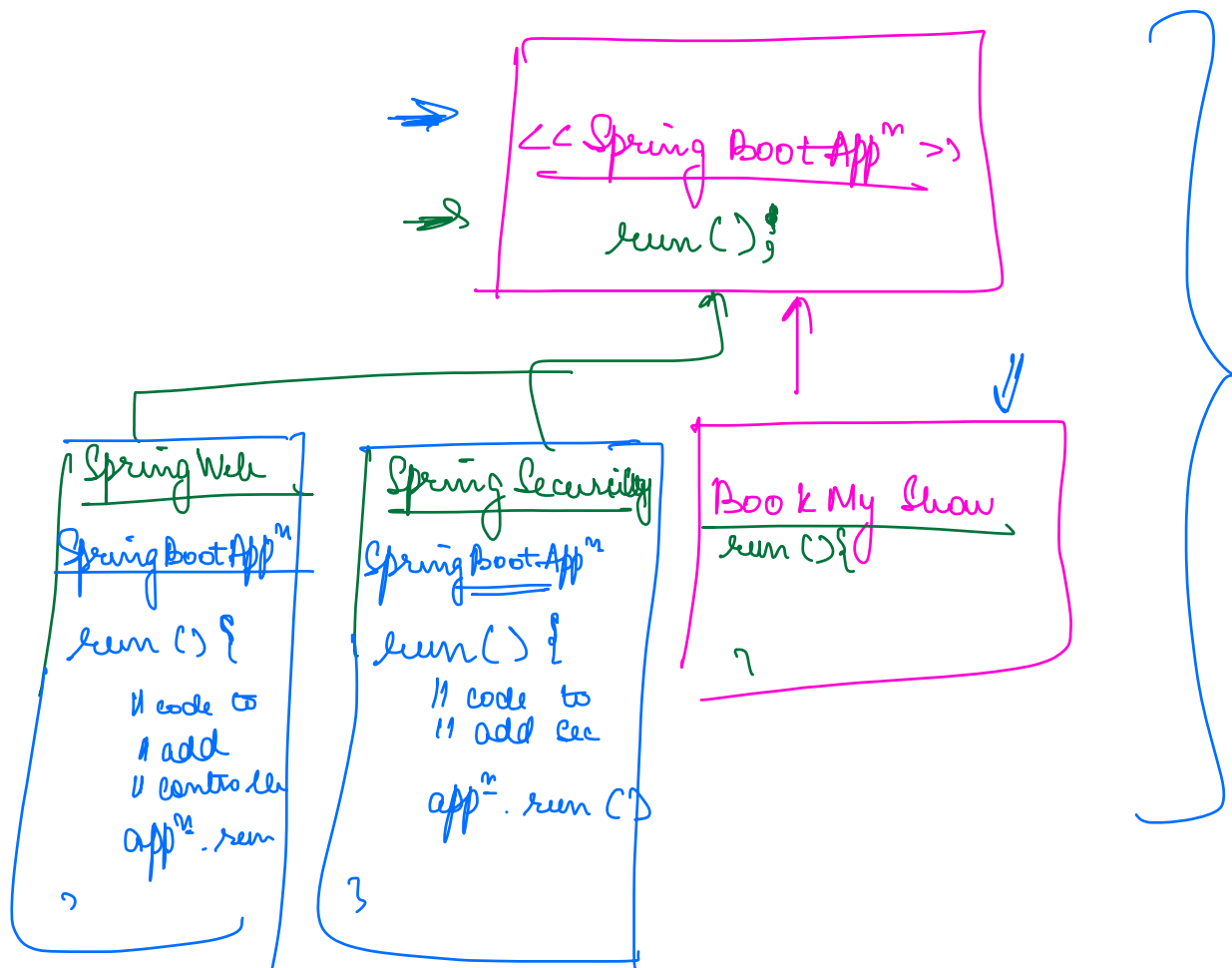
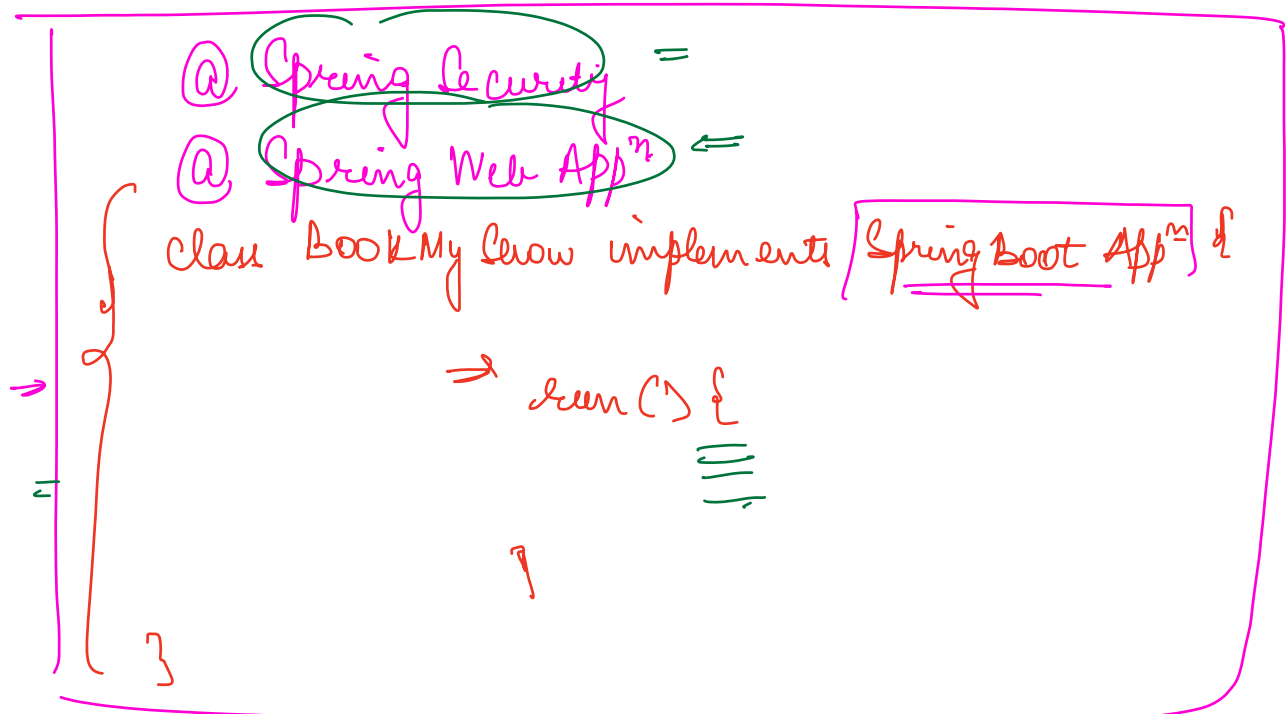
Vanilla + Orange cone + "Chocolate"



C. get Description() : C. get Price()



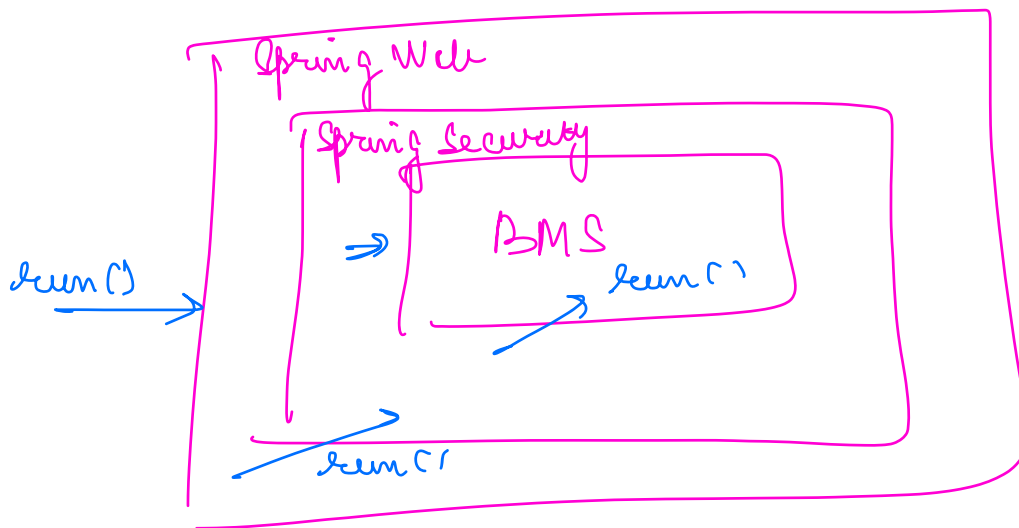
⇒ Very good in iteratively building an object.



@ Spring Web

@ Spring Security

```
class BMC implements Spring Boot App {  
    run() {  
  
    }  
}
```

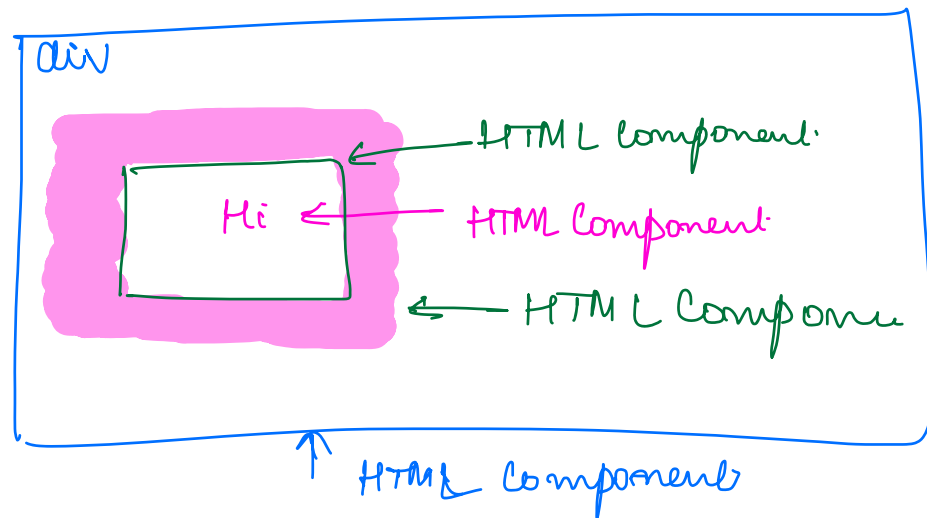
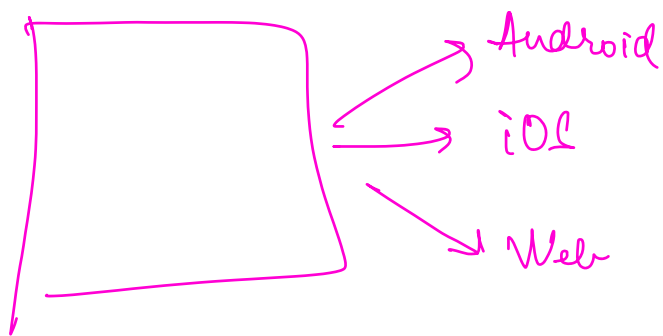


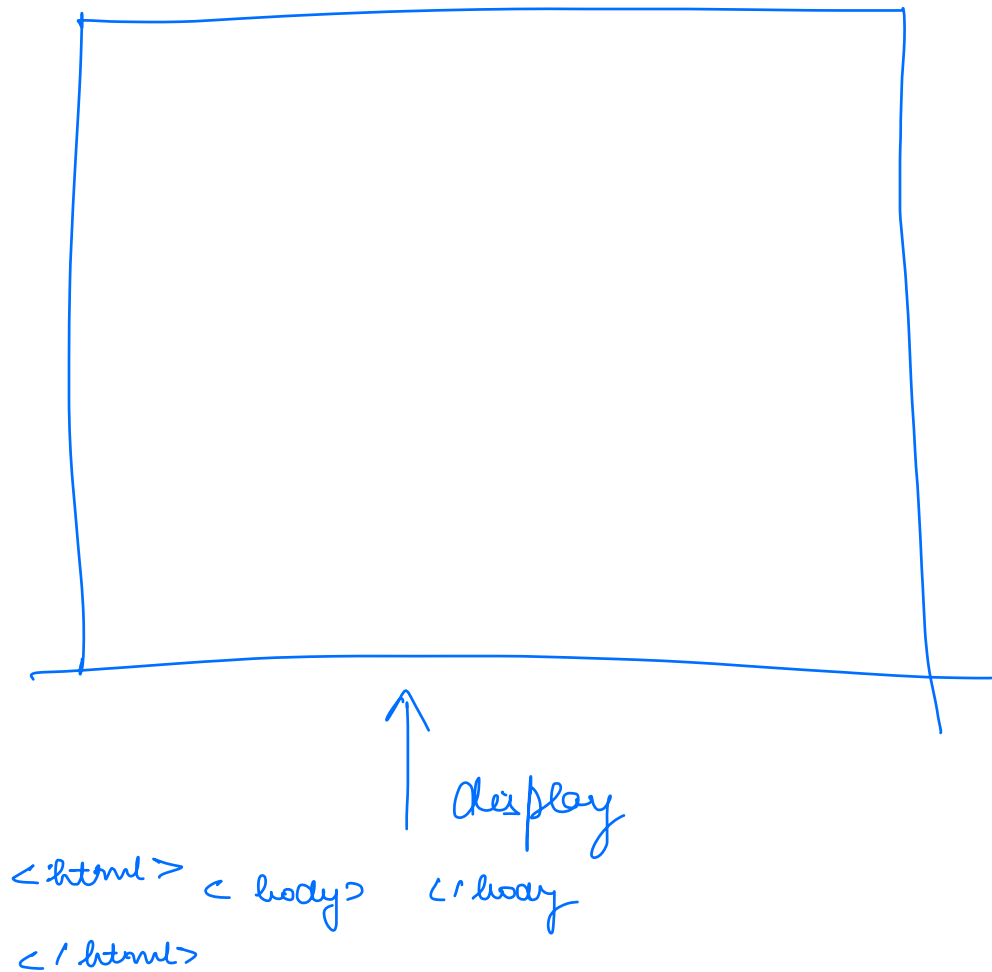
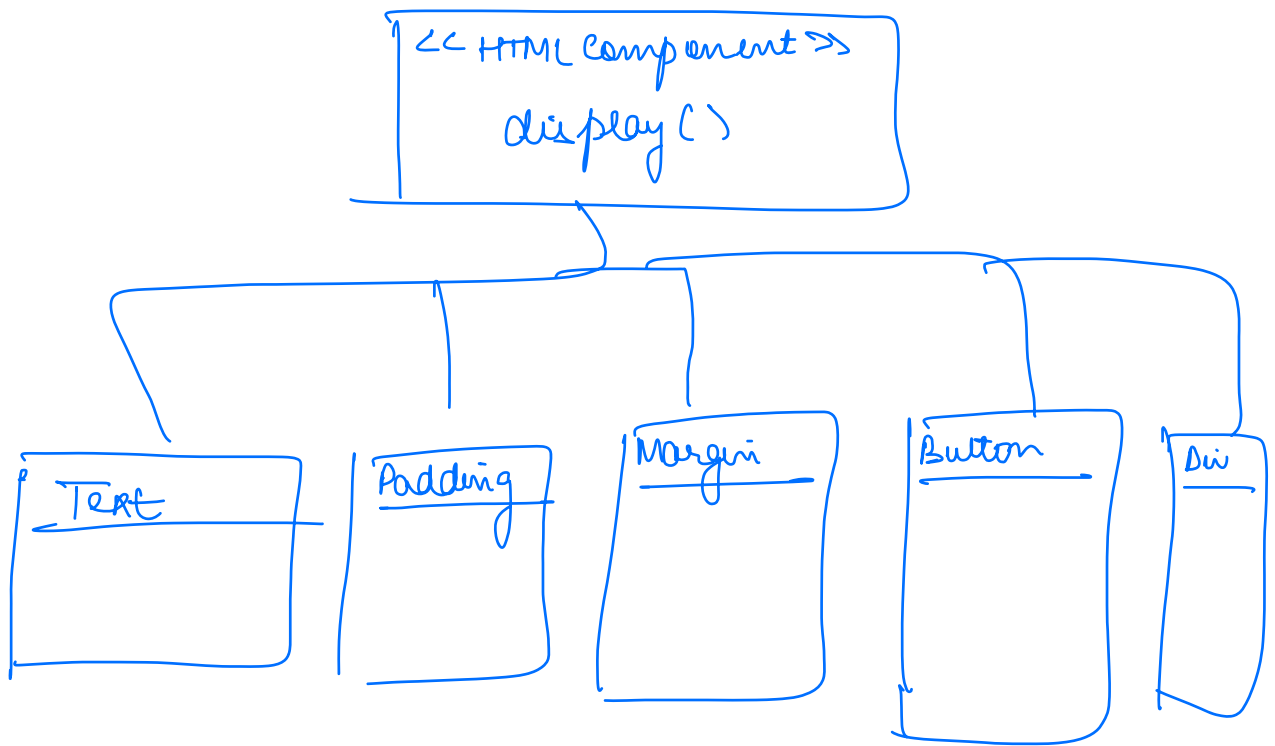
⇒ when at runtime, iteratively I have to add to the behaviour of an entity

Build a Frontend $\Rightarrow$ Flutter / React Native

Cross Platform App^m Dev Framework

Code Once : Run Anywhere



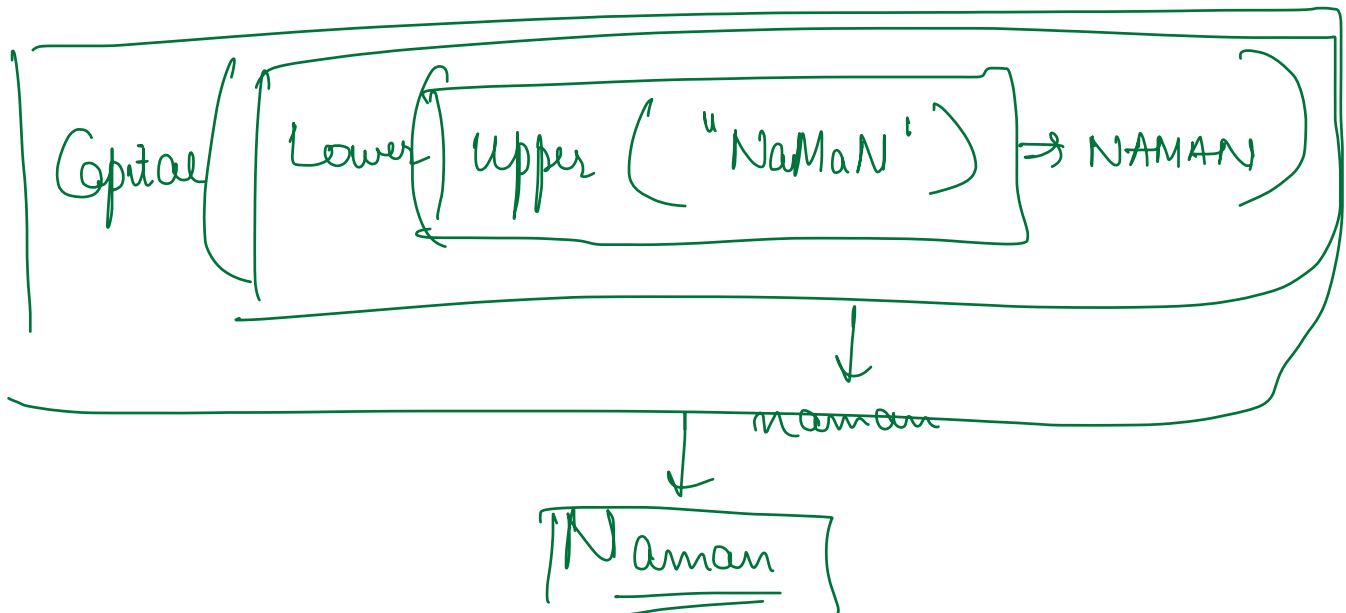


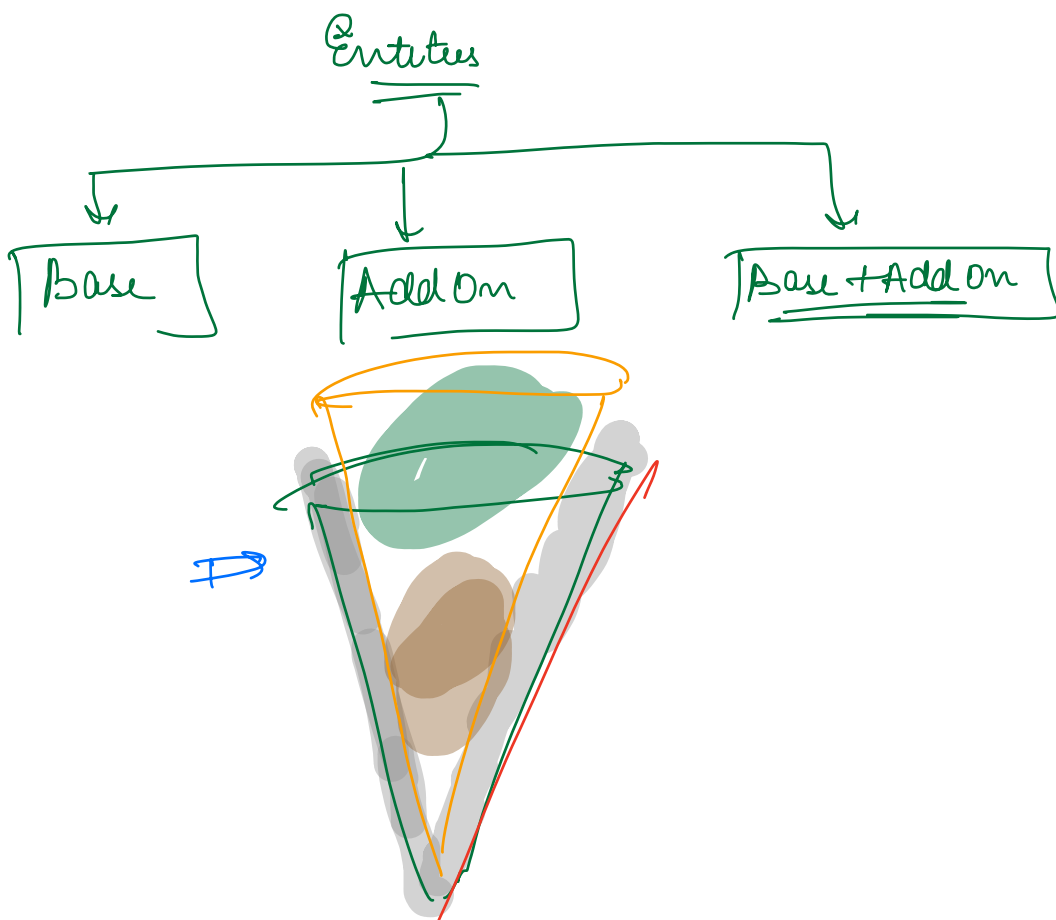
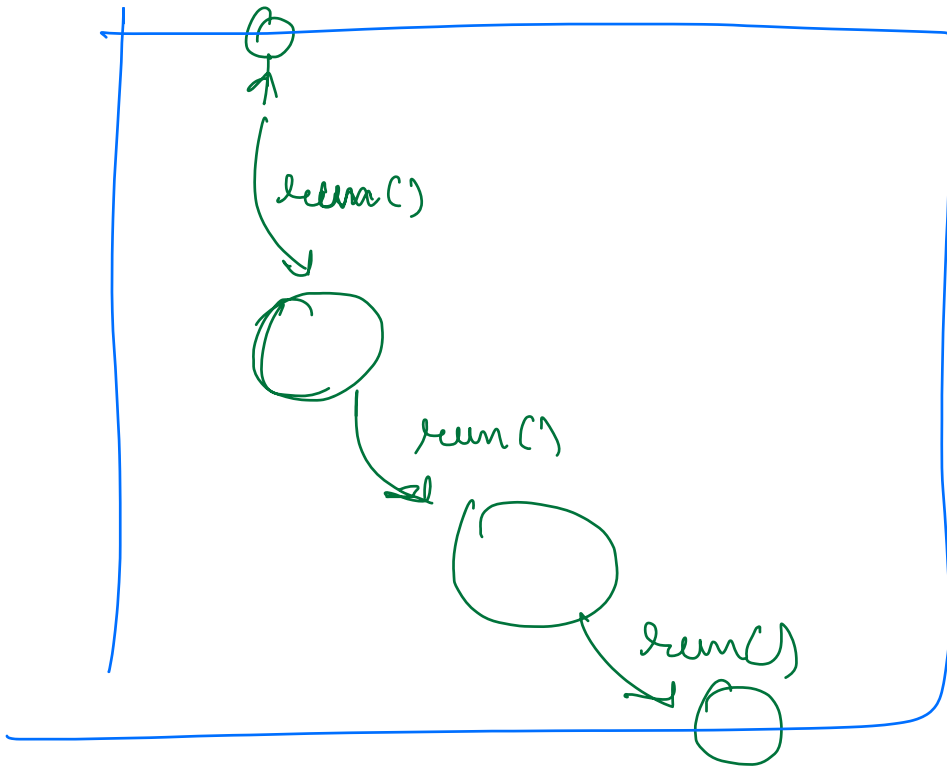
⇒ when you are incrementally building an object where each addition adds to the existing functionalities.

Assg³ Text Formatter

Text.print()

Build an ability to at runtime format how text will look





Class OrangeCone implements IceCreamCone {
IceCreamCone cone;

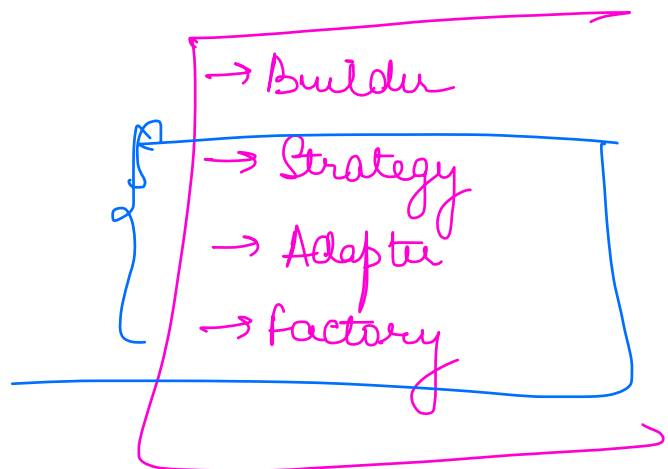
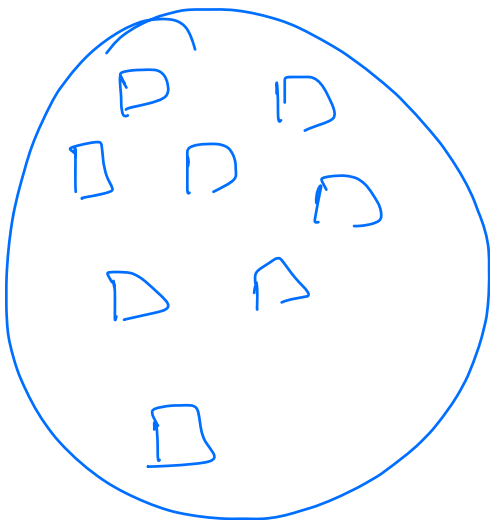
→ OrangeCone (IceCreamCone cone) {
}

⇒ OrangeCone() { }

getPrice() {
if (cone == null) {
return 20

}

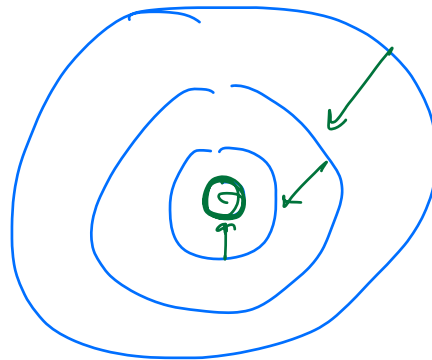
return 20 + cone.getPrice();
}



```
new Orange Cone(  
  new VanillaScoop(  
    new Orange Cone()  
  )  
);
```



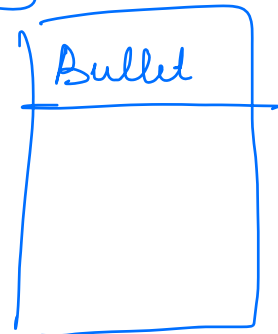
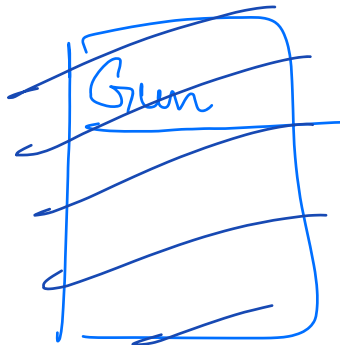
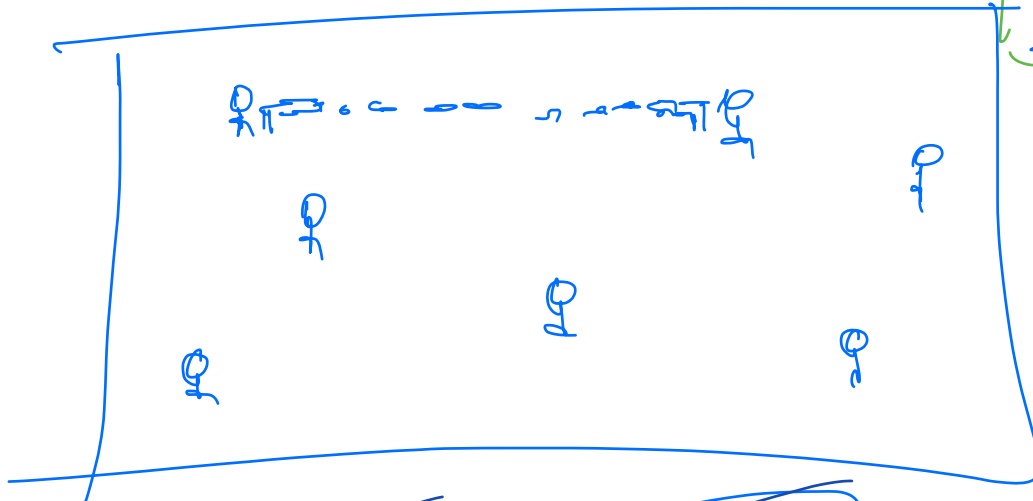
```
new VanillaScoop( )
```

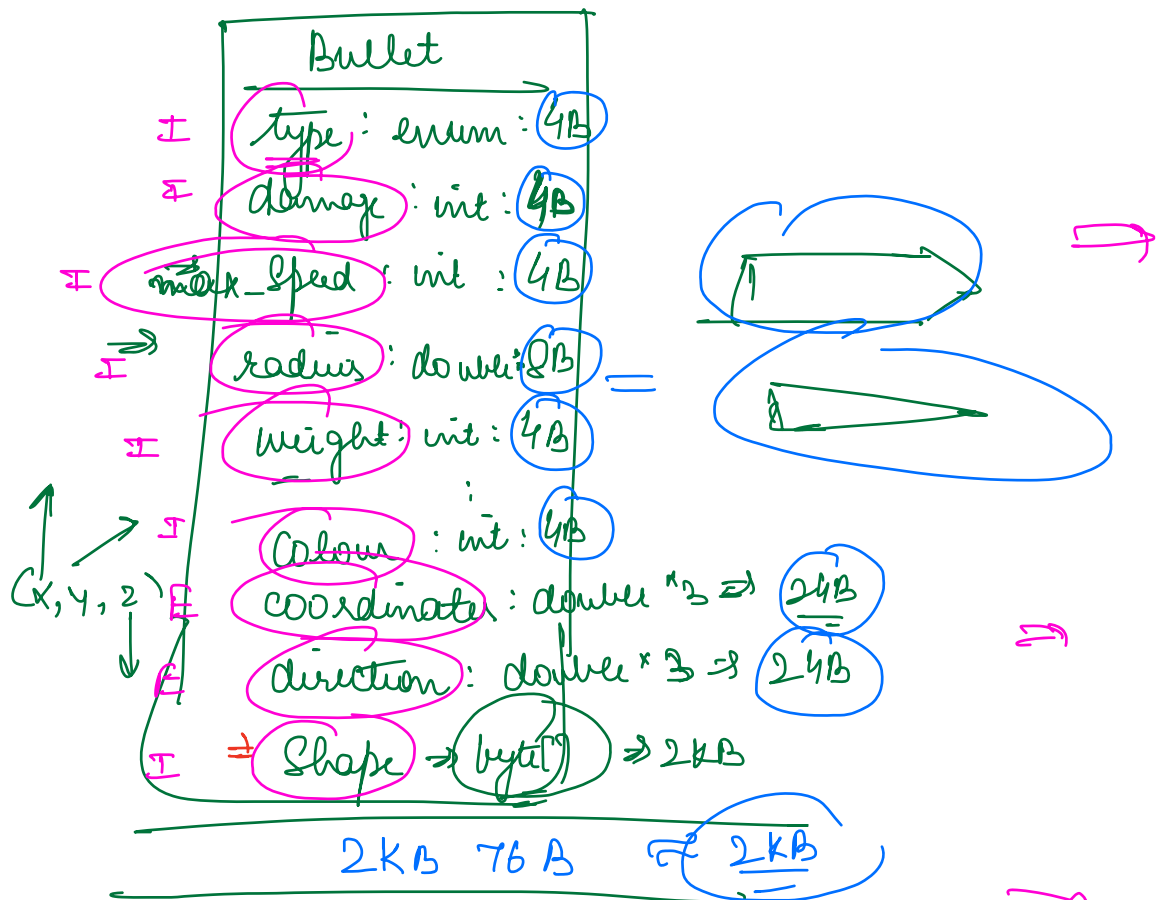


Flyweight Design Pattern



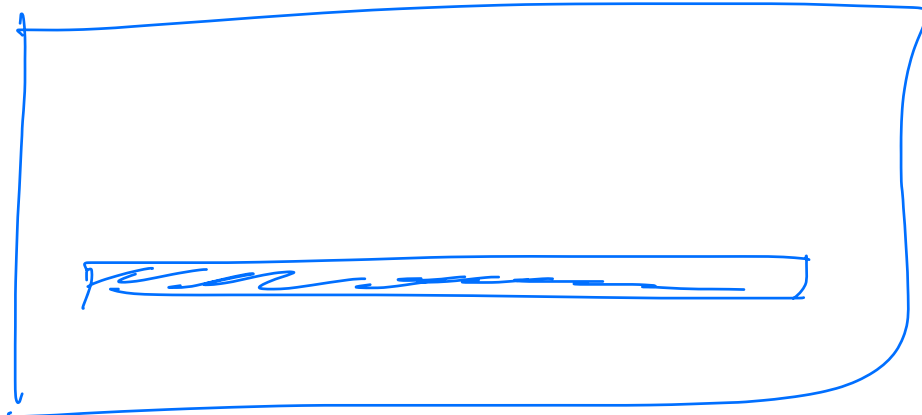
Build / frontend of PulG



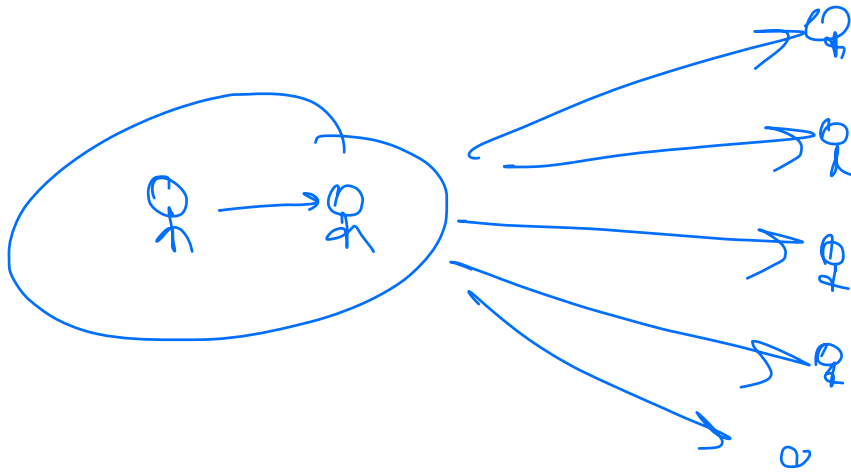


Online games work as follows:

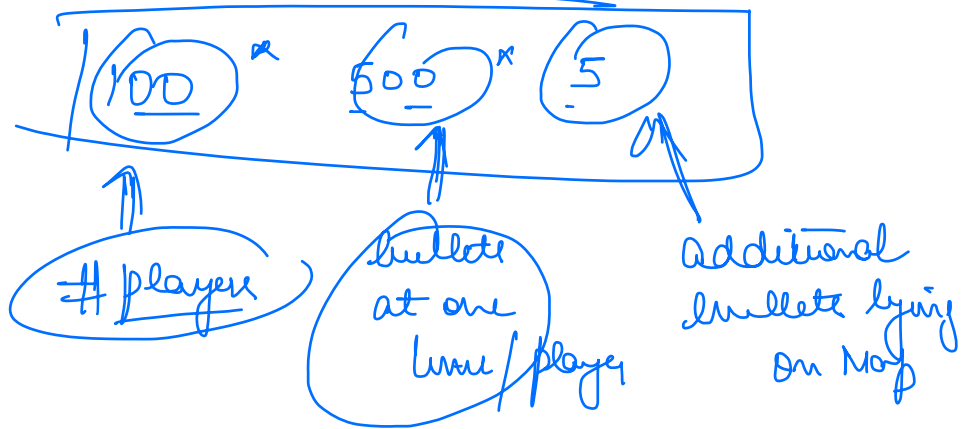
- ① they do unload the complete state of game on the machine of every one



② Whenever any state change happens, it is transferred to every user's device



1 game $\Rightarrow$ how many bullets



$\Rightarrow 25 \times 10^4$ Bullet objects in memory
 size of 1 bullet obj $\approx 2 \text{ KB}$

$\Rightarrow$ to store all Bullets $\Rightarrow 250 \times 10^3 \times 2 \times 10^3 \text{ B}$
 $\Rightarrow 500 \times 10^6 \text{ B}$

⇒ $500 \times 10^3 \text{ KB}$

⇒ $\sqrt{500 \text{ MB}}$
⇒ 0.5 GB

⇒ every type of bullet has a diff shape, not every bullet

Pyweight Design Pattern

→ often we end up with classes with 2 types of properties:

(i) extrinsic

(ii) intrinsic

⇒ that will change for every obj

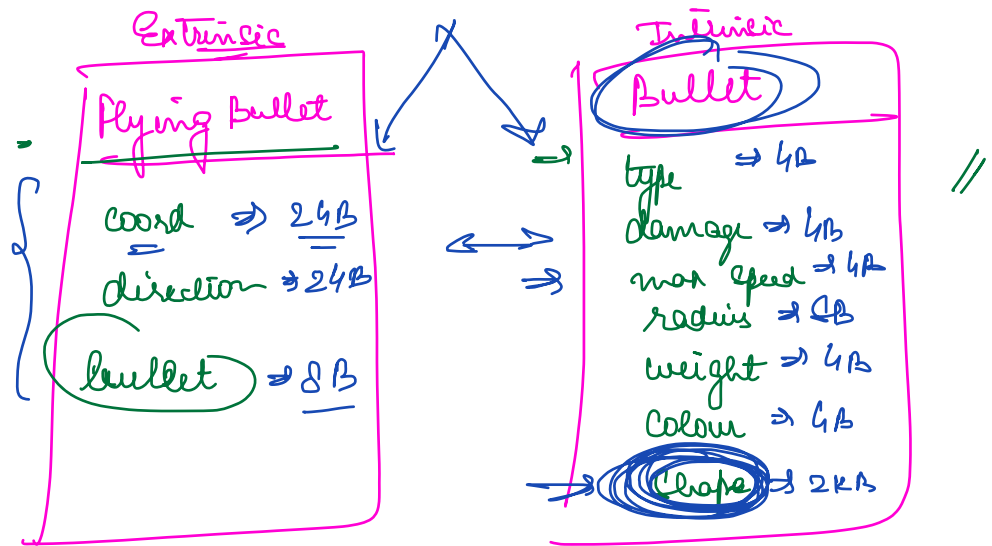
⇒ that will not change (same for many obj)

in some case presence of intrinsic properties in every obj leads to extra memory usage/waste.

⇒ divide the class into 2 diff classes.
One with extrinsic properties

One with intrinsic properties.

⇒ Extrinsic class has a ref to intrinsic



(Each Bullet)

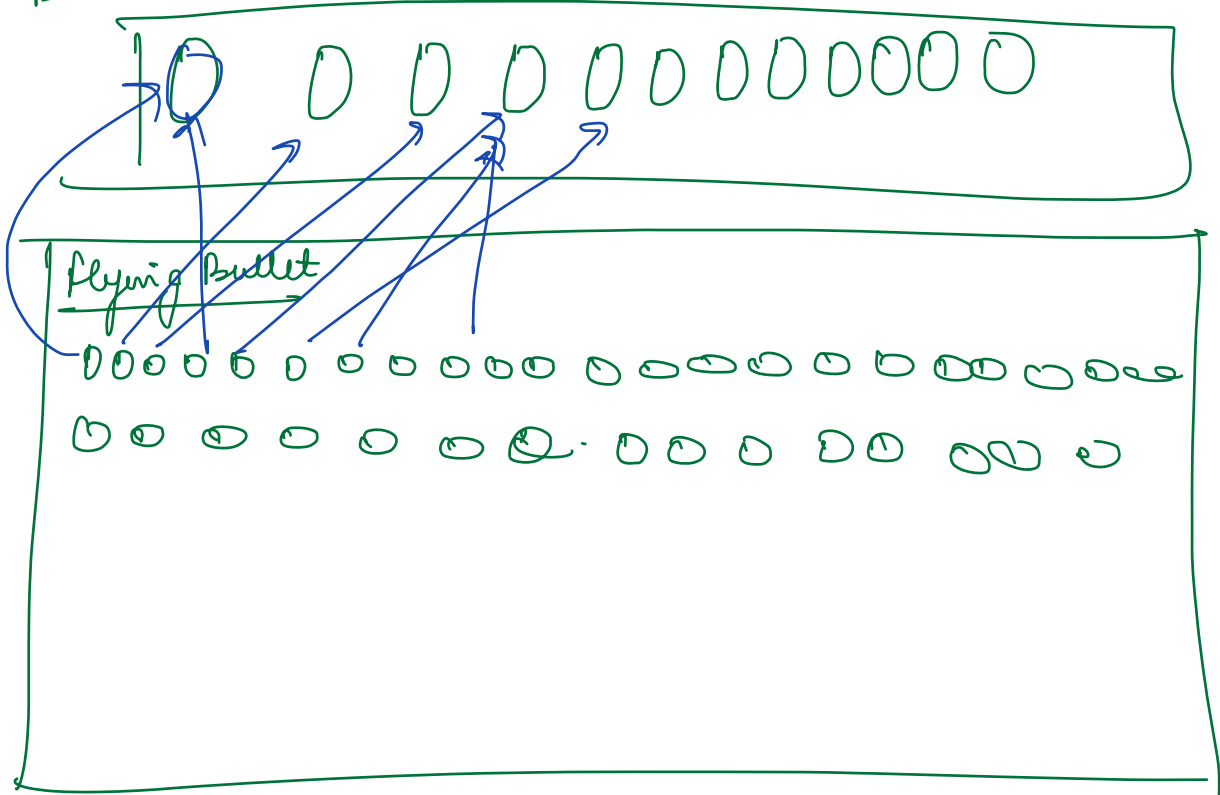
56B

$\Rightarrow$ 250k Flying Bullet

(diff types of Bullet)

2KB

Bullet



$$\text{Total Mem} \Rightarrow (2KB \times 12) + (56 \times 250 \times 10^3)B$$

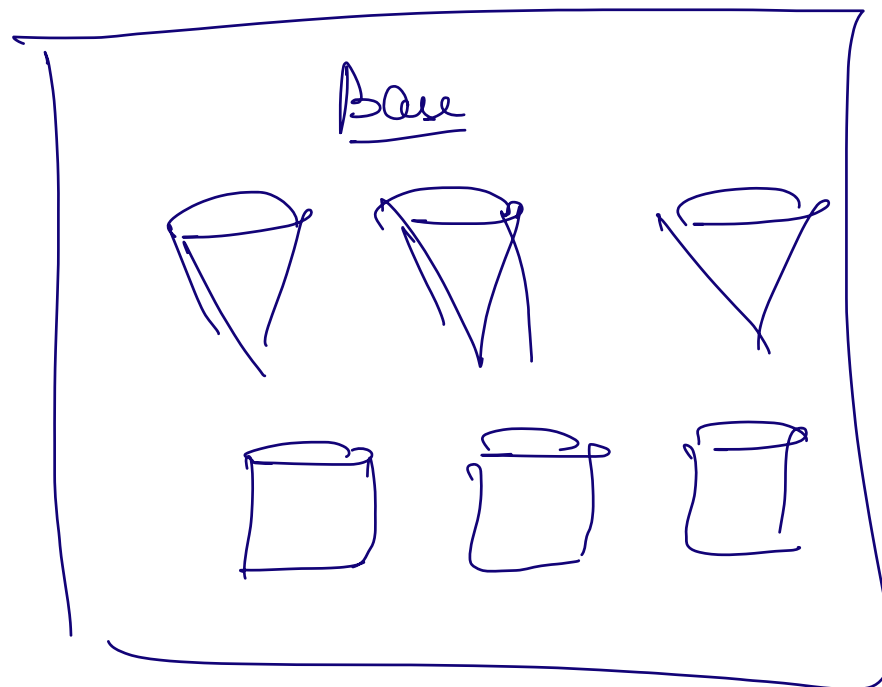
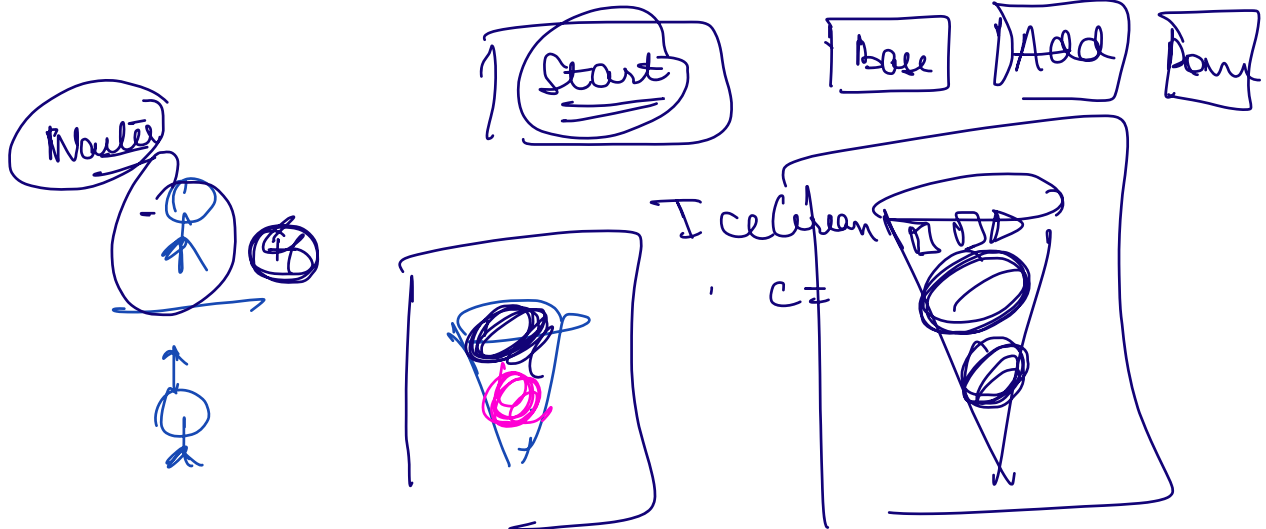
$$\Rightarrow 24 \times 10^3 B + 14000 \times 10^3 B$$

⇒ $1424 \times 10^3 B$

⇒ $1424 KB \Rightarrow 1.4 MB$

from

500MB



		Add Ons	
